# DATABASE DESIGN

# DATABASE_DESIGN.md

## Overview

Relational PostgreSQL database with three tables: `transactions`, `merchant_cache`, and `insights_history`. Relational modeling fits this project well because the entities have clear, fixed relationships (a transaction belongs to a categorization result; insights summarize a set of transactions) — this is not document-shaped, loosely-structured data.

## Tables

### `transactions`

Stores every uploaded transaction and its computed categorization/emissions.

| Column | Type | Notes |
|---|---|---|
| `id` | SERIAL / BIGSERIAL PRIMARY KEY | Surrogate key |
| `upload_batch_id` | UUID | Groups transactions from the same CSV upload |
| `transaction_date` | DATE | From source CSV |
| `raw_description` | TEXT | Original merchant string, before cleaning |
| `cleaned_description` | TEXT | After regex/normalization |
| `amount` | NUMERIC(12,2) | Transaction amount |
| `currency` | VARCHAR(3) | e.g. `INR` |
| `category` | VARCHAR(50) | Final assigned category |
| `categorization_tier` | SMALLINT | Which tier (0–5) produced the category, for transparency/debugging |
| `emission_factor_used` | NUMERIC(10,4) | Factor applied, for auditability |
| `estimated_emissions_kgco2e` | NUMERIC(12,4) | Computed footprint contribution |
| `created_at` | TIMESTAMP WITH TIME ZONE DEFAULT now() | |

### `merchant_cache`

Caches the category resolved for a given merchant string so repeat merchants skip expensive Tier 3–5 lookups.

| Column | Type | Notes |
|---|---|---|
| `id` | SERIAL PRIMARY KEY | |
| `merchant_key` | TEXT UNIQUE | Normalized merchant string used as the cache key |
| `category` | VARCHAR(50) | Cached result |
| `resolved_tier` | SMALLINT | Which tier originally resolved it |
| `confidence_score` | NUMERIC(5,4) | From classifier probability / fuzzy or cosine similarity |
| `hit_count` | INTEGER DEFAULT 1 | Incremented on each cache hit — useful for later analysis of which merchants are most common |
| `last_used_at` | TIMESTAMP WITH TIME ZONE | |

### `insights_history`

Stores LLM-generated insight text per upload batch/period so insights aren't regenerated (and re-billed/re-latency'd) unnecessarily.

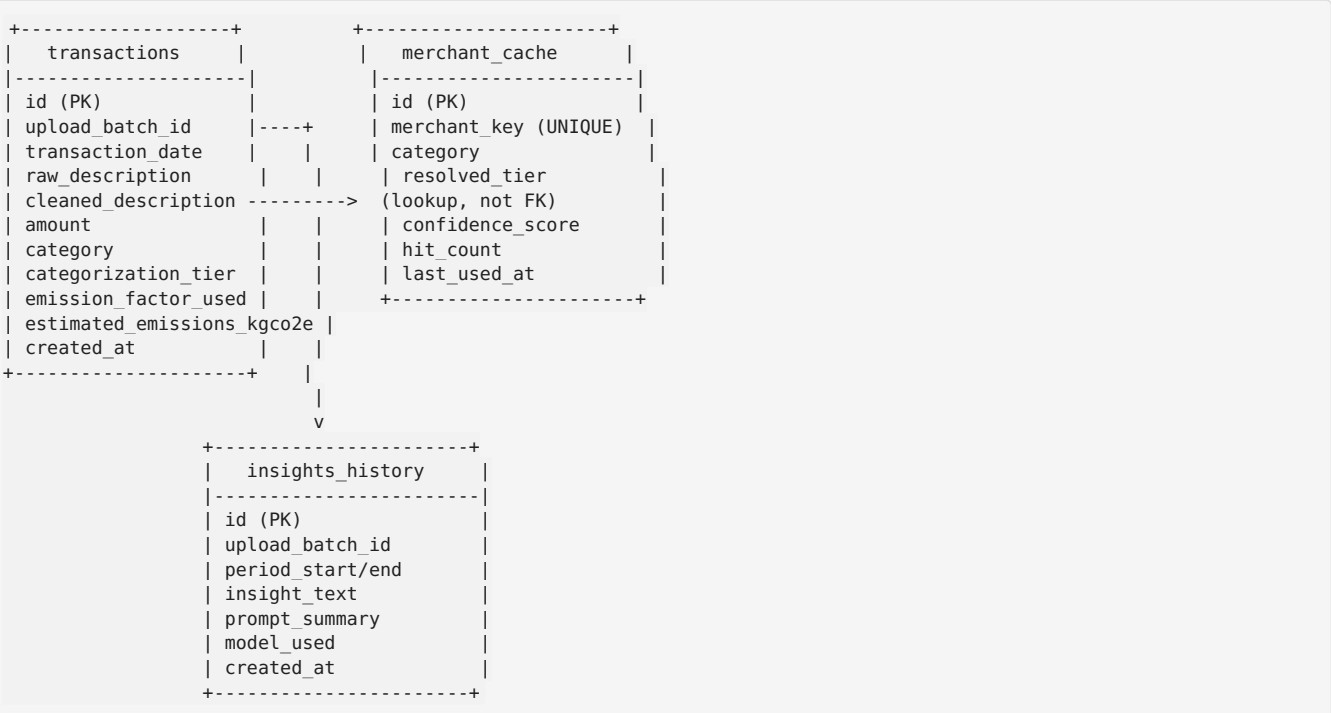
| Column | Type | Notes |
|---|---|---|
| `id` | SERIAL PRIMARY KEY | |
| `upload_batch_id` | UUID | References the batch this insight was generated for |
| `period_start` | DATE | |
| `period_end` | DATE | |
| `insight_text` | TEXT | LLM output |
| `prompt_summary` | TEXT | The aggregated data summary that was sent to the LLM, stored for debugging/reproducibility |
| `model_used` | VARCHAR(50) | e.g. `gemini-1.5-flash` |
| `created_at` | TIMESTAMP WITH TIME ZONE DEFAULT now() | |

## Relationships

```
transactions.upload_batch_id  ---- (many transactions per batch) ---->  insights_history.upload_batch_id
merchant_cache.merchant_key   ---- (looked up by, not FK) ---->  transactions.cleaned_description
```

`merchant_cache` is intentionally **not** a strict foreign key relationship to `transactions` — it's a lookup/cache table keyed by merchant string, independent of any specific transaction row. `insights_history` relates to `transactions` loosely via `upload_batch_id` (not a hard FK in MVP, since insights summarize a batch rather than a single row) — this could be tightened later if needed.

## ER Diagram (ASCII)




## Indexes

- `transactions(upload_batch_id)` — every dashboard query filters by batch.
- `transactions(transaction_date)` — needed for trend charts.
- `transactions(category)` — needed for category breakdown aggregation.

- `merchant_cache(merchant_key)` — already unique, so this is the primary lookup index; unique constraint doubles as an index.
- `insights_history(upload_batch_id)` — lookup insights per batch.

## Validation

- `amount` and `estimated_emissions_kgco2e` constrained via application-level validation (Pandas) before insert, not DB constraints, to keep error messages user-friendly — though a `CHECK (amount IS NOT NULL)` style constraint is reasonable as a backstop.
- `category` should reference a known category list; for MVP this is enforced in application code rather than a DB-level ENUM, to keep it easy to add categories without a migration.

## Normalization Considerations

The schema is in 3NF for the MVP's needs — no repeating groups, no transitive dependencies. A future normalization step could split `category` into its own `categories` table (with metadata like display name, color, default emission factor) once category management becomes more complex than a fixed list — flagged as a V2 consideration.

## Example Rows

**transactions**

```
id | upload_batch_id | transaction_date | cleaned_description | amount | category   | categorization_tier |
estimated_emissions_kgco2e
1  | b3f1...         | 2026-06-03       | swiggy bangalore    | 450.00 | Food Delivery | 2                 | 3.15
```

**merchant_cache**

```
id | merchant_key      | category       | resolved_tier | confidence_score | hit_count
1  | swiggy bangalore  | Food Delivery  | 2             | 0.94             | 27
```

**insights_history**

```
id | upload_batch_id | period_start | period_end | insight_text (truncated)                    | model_used
1  | b3f1...         | 2026-06-01   | 2026-06-30 | "Your food delivery spend implies..."       | gemini-1.5-flash
```

## Future Scalability

- **Connection pooling:** Use PgBouncer (or the hosting provider's built-in pooler, e.g. Supabase's pooled connection string) once concurrent Streamlit sessions exceed the free tier's direct connection limit.
- **Read replicas:** Only relevant at real multi-user production scale — not needed for MVP.
- **Local development:** Run a local Postgres via Docker (`docker run postgres`) to mirror production rather than developing against SQLite, so behavior matches deployment exactly.